

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	P.KALYANI	Reg. No.:	192472199
Section / Batch:	CSE-AI	Date:	8-8-26

Code of Conduct

I **P.KALYANI (192472199)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Kalyani

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

```
def parse_cfg(sentence):
    words = sentence.split()

    # Sentence must have exactly 5 words
    if len(words) != 5:
        return "Invalid Sentence"

    det = ["the", "a"]
    noun = ["student", "teacher", "book"]
    verb = ["reads", "likes"]

    # Grammar:
    #  $S \rightarrow NP VP$ 
    #  $NP \rightarrow Det N$ 
    #  $VP \rightarrow V NP$ 

    if (words[0] in det and
        words[1] in noun and
        words[2] in verb and
        words[3] in det and
        words[4] in noun):
        tree = {
            "S": {
                "NP": {
                    "Det": words[0],
                    "N": words[1]
                },
                "VP": {
                    "V": words[2],
                    "NP": {
                        "Det": words[3],
                        "N": words[4]
                    }
                }
            }
        }
```

```

    }

    }

    }

    return tree

else:

    return "Invalid Sentence"

# Main Program

sentence = input("Enter a sentence: ")

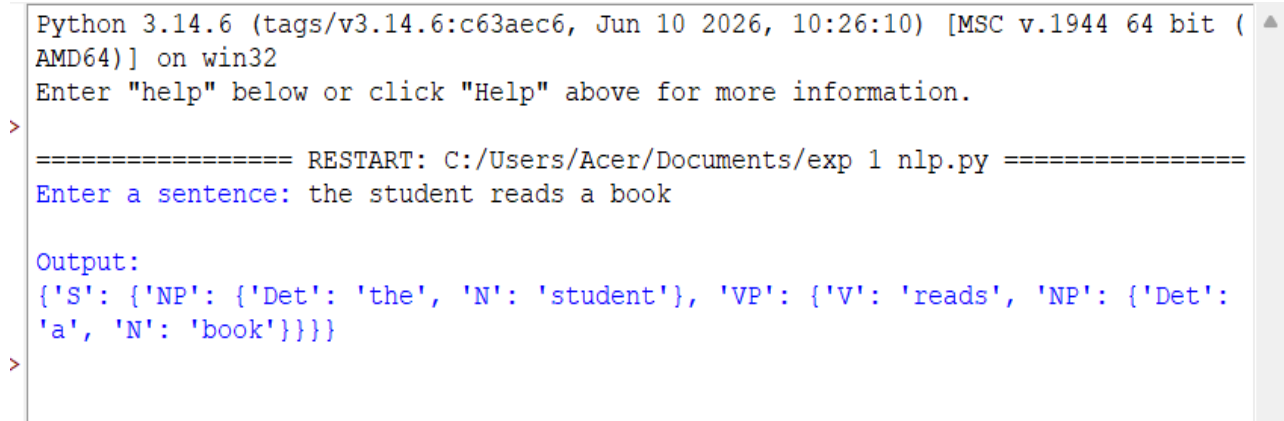
result = parse_cfg(sentence)

print("\nOutput:")

print(result)

```

Output:



```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>
===== RESTART: C:/Users/Acer/Documents/exp 1 nlp.py =====
Enter a sentence: the student reads a book

Output:
{'S': {'NP': {'Det': 'the', 'N': 'student'}, 'VP': {'V': 'reads', 'NP': {'Det': 'a', 'N': 'book'}}}}
>

```

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

```

def check_agreement(subject, verb):

    subjects = {

        "he": "singular",

        "she": "singular",

        "it": "singular",

        "they": "plural"

    }

    verbs = {

        "runs": "singular",

        "writes": "singular",

        "run": "plural",

```

```

"write": "plural"
}

if subject not in subjects or verb not in verbs:

    return False

return subjects[subject] == verbs[verb]

```

Main Program

```

subject = input("Enter subject: ")
verb = input("Enter verb: ")
print(check_agreement(subject, verb))

```

Output:

```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/Acer/Documents/exp 2 nlp.py =====
Enter subject: he
Enter verb: runs
True

```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

```

def sentence_probability(sentence):

    words = sentence.split()

    if len(words) != 2:

        return 0.00

    np = {

        "John": 0.6,

        "Mary": 0.4

    }

    vp = {

```

```
"runs": 0.5,
```

```
"walks": 0.5
```

```
}
```

```
if words[0] in np and words[1] in vp:
```

```
    probability = 1.0 * np[words[0]] * vp[words[1]]
```

```
    return round(probability, 2)
```

```
return 0.00
```

```
# Main Program
```

```
sentence = input("Enter sentence: ")
```

```
print("Probability =", sentence_probability(sentence))
```

Output:

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
```

```
Enter "help" below or click "Help" above for more information.
```

```
===== RESTART: C:/Users/Acer/Documents/exp nlp 3.py =====
```

```
Enter sentence: John runs
```

```
Probability = 0.3
```

```
|
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	----	---	--	--	----------------------------------

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____